

Buffer output

- Le socket TCP hanno un buffer per l'output (send buffer) in cui vengono collocati temporaneamente i dati da trasmettere
- La dimensione di questo buffer può essere configurata mediante l'opzione `SO_SNDBUF`
- Se si chiama `write()` con `n` byte sul socket TCP, il kernel cerca di copiare `n` byte sul buffer del socket. Se il buffer del socket è più piccolo di `n` byte o parzialmente occupato da dati non ancora trasmessi verranno copiati un numero minore di byte
- Se il socket è bloccante alla fine della `write` saranno inviati i byte indicati dal valore di ritorno della `write`

Socket

- **Lettura e scrittura safe**

```
#include <sys/types.h>
#include <sys/socket.h>
#include <errno.h>

ssize_t recv_all(int fd, void *buf, size_t n) {
    size_t received = 0;
    char *p = buf;

    while (received < n) {
        ssize_t r = recv(fd, p + received, n - received, 0);

        if (r < 0) {
            if (errno == EINTR)
                continue;    // riprova
            return -1;        // errore
        }
        if (r == 0) {
            // connessione chiusa -> restituisco quanti byte ho letto
            return received;
        }
        received += (size_t)r;
    }
    return (ssize_t)received; // = n
}
```

Socket

- **Lettura e scrittura safe**

```
#include <sys/types.h>
#include <sys/socket.h>
#include <errno.h>

ssize_t send_all(int fd, const void *buf, size_t n) {
    size_t sent = 0;
    const char *p = buf;

    while (sent < n) {
        ssize_t w = send(fd, p + sent, n - sent, 0);

        if (w < 0) {
            if (errno == EINTR)
                continue;    // riprova
            return -1;        // errore
        }

        sent += (size_t)w;
    }

    return (ssize_t)sent;    // = n
}
```

Comunicazione senza connessione socket UDP

UDP (User Datagram Protocol) è un protocollo di trasporto semplice. Se TCP è orientato alla connessione, UDP non stabilisce connessioni tra client e server e non offre garanzie di affidabilità.

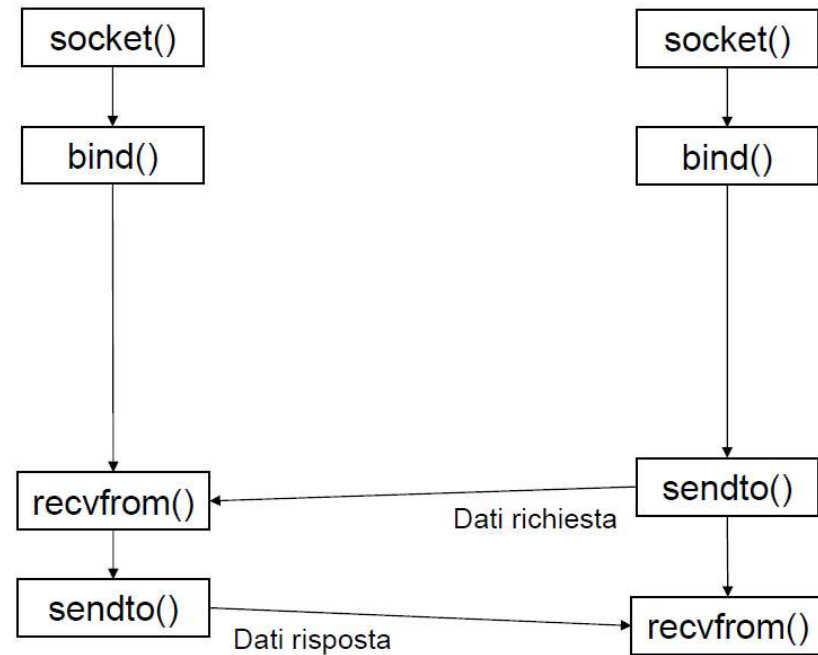
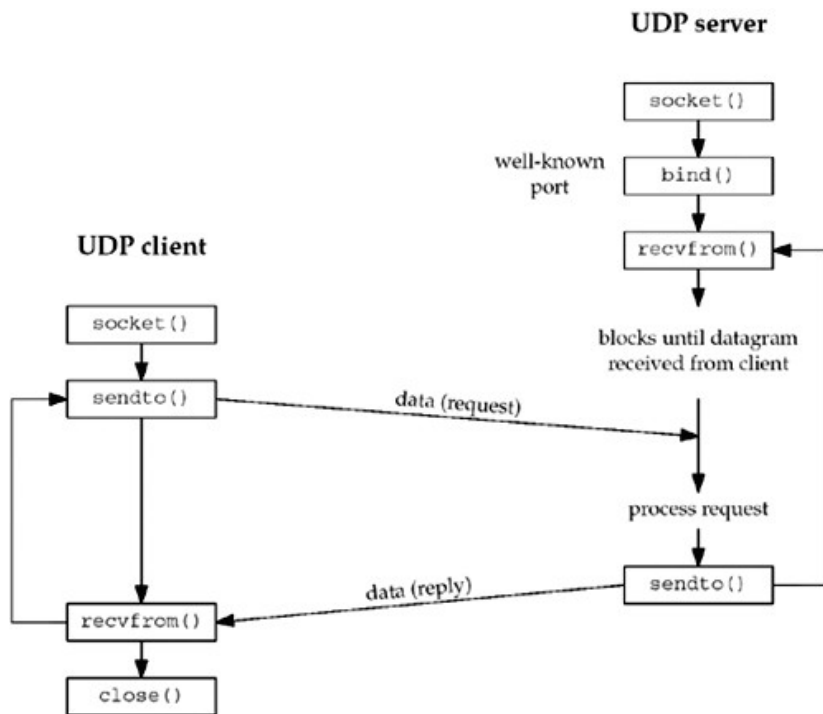
Se TCP trasmette flussi di byte, il protocollo UDP invia pacchetti di taglia fissa, detti datagram.

L'assenza di connessione (protocolli di handshake in 3 passi e 4 passi di chiusura) rende UDP più veloce di TCP per trasferimenti di pacchetti di byte.

Gli errori di trasmissione sono sporadici quindi se i dati non sono critici la comunicazione può essere molto rapida ed efficace

Comunicazione senza connessione socket UDP

```
(sockfd = socket(AF_INET, SOCK_DGRAM, 0))
```



Funzione sendto()

```
int sendto(int s, const void *msg, int len,  
           unsigned int flags,  
           const struct sockaddr *to, int tolen);
```

- Trasmissione non affidabile:
 - Non errore se pacchetto non raggiunge l'host remoto
 - Solo errori locali (e.g. dimensione pacchetto troppo grande EMSGSIZE)
- Parametri:
 - s: descrittore socket
 - msg: puntatore al buffer del messaggio
 - len: dimensione del pacchetto
 - to: indirizzo del destinatario
 - tolen: dimensione della struttura indirizzo

Funzione recvfrom()

```
int recvfrom(int s, void *buf, int len,  
             unsigned int flags,  
             struct sockaddr *from,  
             int *fromlen);
```

- Restituisce byte ricevuto, -1 se errore:
- Parametri:
 - s: descrittore socket
 - msg: puntatore al buffer del messaggio
 - len: dimensione del buffer (se dimensione minore del pacchetto si leggono i primi len byte)
 - from: indirizzo dell'end point mittente
 - fromlen: puntatore alla dimensione (byte) della struttura sockaddr
 - Con flag MSG_PEEK non toglie pacchetto dalla coda e verifica solo indirizzo client
 - Se non arrivano messaggi rimane bloccato, si può settare timeout

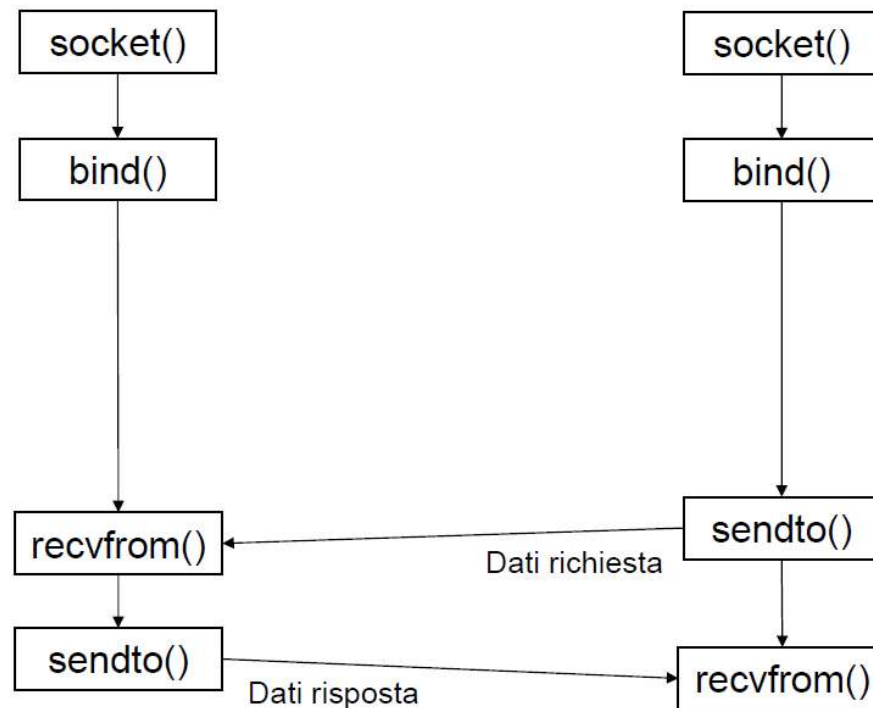
Uso di connect()

- connect() si può usare anche per comunicazione senza connessione
 - Per esempio per impostare la connessione una volta per tutte e gestire presenza di errori sulla connessione
- Se è stabilita la connect() si può usare write, send o sendto lasciando NULL gli indirizzi:

```
int sendto(sd, buf, len, flags, NULL,0)
```


Comunicazione senza connessione

- Con flusso di dati senza connessione (SOCK_DGRAM) server e client faranno `bind()` su indirizzo locale per poi comunicare con `sendto()/recvfrom()`



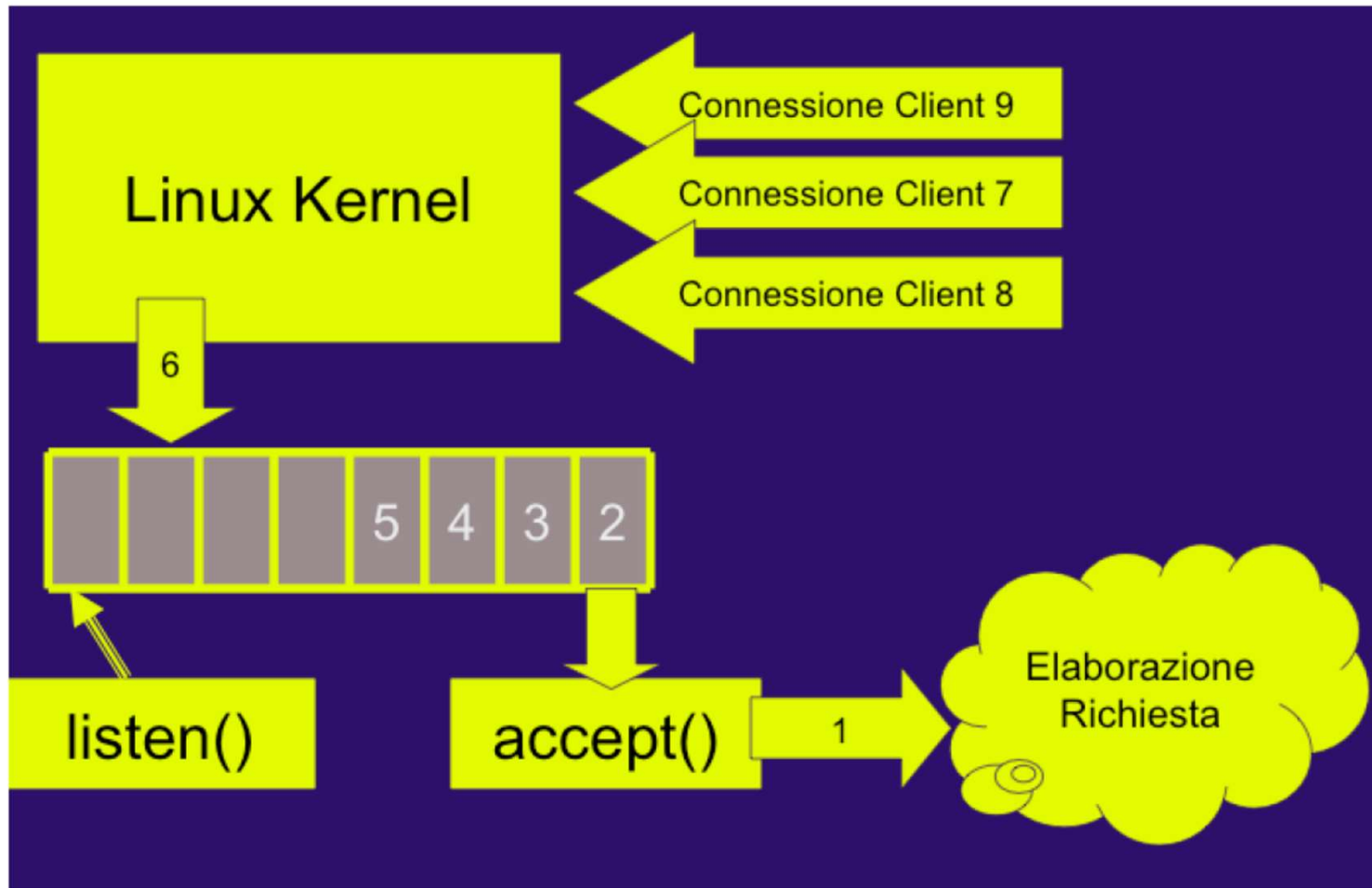
Server Concorrente

- **Generico server**

- Un generico server attende le richieste di connessione su una determinata porta
- Possono arrivare richieste concorrenti e da molteplici client
- Una soluzione senza programmazione concorrente:
 - client serviti uno alla volta, finché una connessione non è terminata, non vengono serviti altri client interessati al servizio
 - N.B. comunque è il SO che gestisce le richieste concorrenti ed in effetti le serializza

Server Concorrente

- Coda delle richieste



Server Concorrente

- **Concorrenza**

- Un server che gestisce sequenzialmente i vari client è insoddisfacente
 - il tempo di attesa di ogni client
 - potrebbe risultare eccessivo
 - dipende dalla durata delle altre connessioni
 - se le connessioni durano molto, il server ben presto diverrebbe indisponibile anche solo ad accodare le nuove richieste di connessioni
- Nei server reali le richieste di vari client devono essere gestite concorrentemente

Server Concorrente

- **Server a programmazione concorrente**
 - se il server è **concorrente**, per ogni client viene generato un processo ad hoc per offrire il servizio, in modo che più client possano essere serviti in modalità concorrente. In tal caso l'attesa sulla coda è limitata al tempo necessario alla gestione della richiesta del client da parte del server ed allo start-up del nuovo processo;

Server Concorrente

- **Server a programmazione concorrente**

```
socket(...);
bind(...);
listen(...);

while (1) {
    fd2 = accept(fd1, (struct sockaddr *)NULL, NULL);

    if ( (pid = fork()) < 0 ) {
        perror("fork");
        exit(1);
    } else if (pid == 0) {    // processo figlio
        close(fd1); // al figlio non serve fd1
        // gestisce la connessione usando fd2
        ...
        exit(0); // poi il figlio termina
    }
    // il processo padre chiude fd2 e ripete il ciclo
    close(fd2);
}
```

Socket

- **Server concorrente**

```
#define PORT 5201
#define BACKLOG 8

static void handle_client(int fd) {
    char buf[1024];
    for (;;) {
        ssize_t n = recv(fd, buf, sizeof(buf), 0);
        if (n == 0) break;           // client ha chiuso
        if (n < 0) { if (errno == EINTR) continue; perror("recv"); break; }
    }
    // echo
    ssize_t off = 0;
    while (off < n) {
        ssize_t w = send(fd, buf + off, (size_t)(n - off), 0);
        if (w < 0) { if (errno == EINTR) continue; perror("send"); }
        close(fd); _exit(1); }
    off += w;
}
close(fd);
_exit(0); // il figlio termina qui
}
```


Socket

■ Server concorrente

```
int main(void) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    if (s < 0) { perror("socket"); return 1; }

    struct sockaddr_in addr = {0};
    addr.sin_family      = AF_INET;
    addr.sin_port        = htons(PORT);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    if (bind(s, (struct sockaddr*)&addr, sizeof(addr)) < 0) { perror("bind"); return 1; }
    if (listen(s, BACKLOG) < 0) { perror("listen"); return 1; }
    printf("Server in ascolto su 0.0.0.0:%d (fork per connessione)\n", PORT);

    for (;;) {
        int c = accept(s, NULL, NULL);
        if (c < 0) { if (errno == EINTR) continue; perror("accept"); continue; }
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); close(c); continue; }
        if (pid == 0) { // FIGLIO
            close(s); // al figlio non serve la listening socket
            handle_client(c); // non ritorna, figli restano zombie
        }
        close(c); // il padre torna ad accettare altri client
    }
}
```


Socket

- **Per evitare zombie**

```
#define PORT 5201
#define BACKLOG 8
```

```
// handler: raccoglie tutti i figli terminati
static void reap(int sig) {
    (void)sig;
    while (waitpid(-1, NULL, WNOHANG) > 0) {}
}
```

```
int main(void) {
    // installa SIGCHLD handler (SA_RESTART riavvia accept/recv interrotti)
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = reap;
    sa.sa_flags = SA_RESTART;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGCHLD, &sa, NULL);
}
```

Socket

- **Esercizi**

- Scrivere il client per comunicare con questo server
- Scrivere la versione multithreaded del server

Socket

- **Server multithreaded**

```
socket(...);
bind(...);
listen(...);

while (1){
    client_len = sizeof(client_addr);
    connect_sd = accept(listen_sd,
                        (struct sockaddr *) &client_addr,
                        &client_len)

    thread_sd = (int *) malloc(sizeof(int));

    *thread_sd = connect_sd;
    printf("server: new connection from %d \n",connect_sd);
    pthread_create(&tid, NULL, gestisci, (void *) thread_sd);
}
```

Socket

■ Server concorrente

```
static void *gestisci(void *arg) {
    int sd = *(int *)arg;
    free(arg);                // non serve più il contenitore

    char buf[1024];
    for (;;) {
        ssize_t n = recv(sd, buf, sizeof(buf), 0);
        if (n == 0) break;    // client ha chiuso
        if (n < 0) { if (errno == EINTR) continue; perror("recv"); break; }

        // echo (gestione parziale minima)
        ssize_t off = 0;
        while (off < n) {
            ssize_t w = send(sd, buf + off, (size_t)(n - off), 0);
            if (w < 0) { if (errno == EINTR) continue; perror("send"); close(sd); return NULL; }
            off += w;
        }
    }
    close(sd);
    return NULL;
}
```

Socket

■ Server concorrente

```
for (;;) {  
    int connect_sd = accept(listen_sd, NULL, NULL);  
    if (connect_sd < 0) { if (errno == EINTR) continue; perror("accept"); continue; }  
  
    // allochiamo un int per passarlo al thread in modo sicuro  
    int *thread_sd = malloc(sizeof(*thread_sd));  
    if (!thread_sd) { perror("malloc"); close(connect_sd); continue; }  
    *thread_sd = connect_sd;  
  
    pthread_t tid;  
    if (pthread_create(&tid, NULL, gestisci, thread_sd) != 0) {  
        perror("pthread_create");  
        close(connect_sd);  
        free(thread_sd);  
        continue;  
    }  
    pthread_detach(tid);    // nessun join: il thread si "auto-raccoglie"  
}  
}
```

- malloc per evitare di sovrascrivere connect
- pthread_detach evita di dover fare pthread_join() (niente thread-zombie)

Socket

■ Opzioni socket

```
int getsockopt(int fd, level, option, void *val, socklen_t len);  
int setsockopt(int fd, level, option, void *val, socklen_t *len);
```

- leggono e scrivono le opzioni di un socket
- le opzioni si dividono in *livelli*, identificati da apposite costanti
- livelli:

livello socket	SOL_SOCKET
livello TCP	IPPROTO_TCP
livello IP	IPPROTO_IP
- il significato di val dipende dall'opzione
- len è pari alla dimensione dell'oggetto puntato da val
- restituiscono: 0 se OK, -1 altrimenti (e impostano errno)

Socket

■ Opzioni socket

- Opzione SO_REUSEADDR

```
int getsockopt(int fd, level, option, void *val, socklen_t len);  
int setsockopt(int fd, level, option, void *val, socklen_t *len);
```

- opzione di livello socket
- permette a bind di assegnare un indirizzo ancora occupato
- val deve puntare ad un intero
 - se l'intero è diverso da zero, questa opzione è attiva
 - se l'intero puntato contiene zero, l'opzione è inattiva
 - len è pari a sizeof(int)

Socket

- **Opzioni socket**

- Opzione `SO_REUSEADDR`

- **Contesto:**

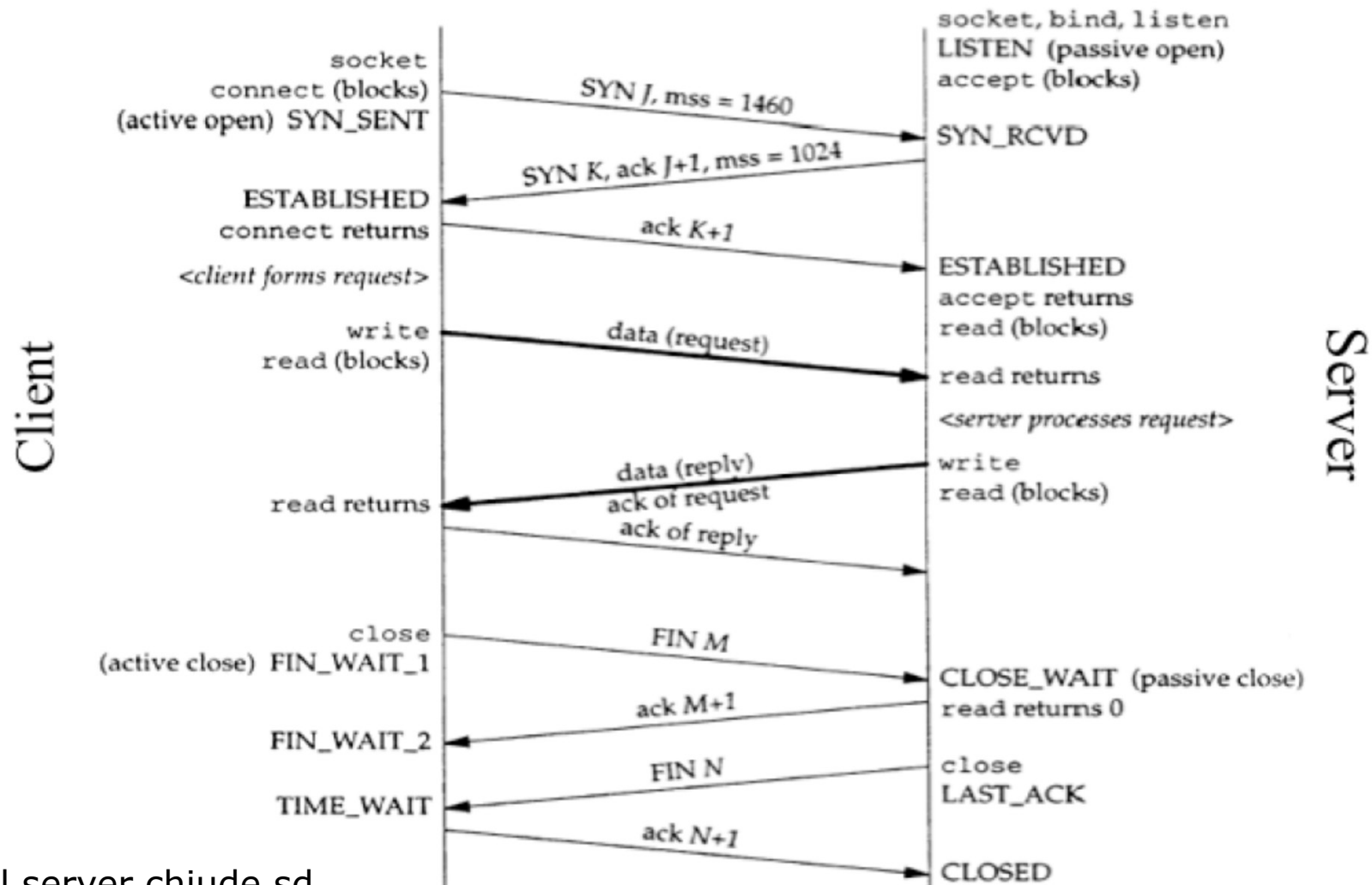
- Quando una connessione TCP si chiude, non viene smontata subito: resta in `TIME_WAIT` per ~1–4 minuti. Serve per:
 - evitare pacchetti “ritardati” che possano confondere una nuova connessione garantire chiusura ordinata
 - Durante `TIME_WAIT`, un nuovo socket NON può bindare quella porta → **ma con `SO_REUSEADDR` sì.**

- **Quando serve:**

- Dopo uso occorre ancora `bind(sock, ...)` su stesso indirizzo
- il server non riesce a ripartire → “address already in use”
- Con l’opzione: parte senza problemi

Socket TCP

- Tipico schema di connessione
 - handshake segmenti TCP



Nota: il server chiude sd
di connect, non di listen

Socket

- **Opzioni socket**

- Opzione SO_REUSEPORT (non standard POSIX)

- **Contesto:**

- Consente di creare **più socket in listen() sulla stessa coppia (IP, PORT)**
- Il kernel distribuisce automaticamente le connessioni tra i vari socket
→ load balancing nativo

- **Quando serve:**

- load balancing tra più thread/processi
- server multicore ad alte prestazioni
- riduzione del lock contention sul singolo socket

Socket

- **Opzioni socket**

- Opzione SO_REUSEPORT

- **Contesto:**

- Consente di creare **più socket in listen() sulla stessa coppia (IP, PORT)**
- Il kernel distribuisce automaticamente le connessioni tra i vari socket.
→ load balancing nativo

```
int fd = socket(AF_INET, SOCK_STREAM, 0);  
int opt = 1;  
setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &opt, sizeof(opt));
```

```
bind(fd, ...);  
listen(fd, ...);
```

- Si può ripetere questo codice in più processi → tutti andranno in listen() sulla stessa IP:PORT.

Socket

- **Opzioni socket**

- Opzione SO_REUSEPORT

- **Contesto:**

- Consente di creare **più socket in listen() sulla stessa coppia (IP, PORT)**
- Il kernel distribuisce automaticamente le connessioni tra i vari socket.
→ load balancing nativo

- **Funzionamento:**

- Il kernel mantiene un "gruppo" di socket compatibili (stesso IP/PORT) e li mette in un pool.
- Poi fa round-robin o hash-based distribution delle nuove connessioni tra i socket.

- **Vantaggio:**

Ogni thread/processo ha il proprio socket → non serve contesa sullo stesso queue → massima scalabilità multicore.

Socket

- **Opzioni socket**

- Opzione SO_REUSEPORT

- **Contesto:**

- Consente di creare **più socket in listen() sulla stessa coppia (IP, PORT)**
- Il kernel distribuisce automaticamente le connessioni tra i vari socket.
→ load balancing nativo

- **Vantaggio:**

Attenzione a differenza tra listen socket e accept socket

- Senza SO_REUSEPORT ogni accept/connect è un canale separato
- Con SO_REUSEPORT, non hai più thread che fanno accept() sullo stesso socket, ma ogni thread/processo ha il suo listening socket, ognuno ha la sua accept queue

Socket

- **Opzioni socket**
 - Opzione SO_REUSEPORT
- **Esempio:** ogni thread

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
int opt = 1;

setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &opt, sizeof(opt));

struct sockaddr_in addr = {
    .sin_family = AF_INET,
    .sin_port   = htons(8080),
    .sin_addr   = { .s_addr = htonl(INADDR_ANY) }
};

bind(fd, (struct sockaddr*)&addr, sizeof(addr));
listen(fd, SOMAXCONN); // massima coda ammessa dal sistema

// poi ogni thread fa accept() sul proprio fd
for(;;) {
    int cfd = accept(fd, NULL, NULL);
    // gestisci la connessione...
}
```

Funzioni bloccanti

- La funzione `accept()` e le funzioni per gestire I/O sono bloccanti (`read`, `recv`)
- Il server ricorsivo tradizionale
 - attende su `accept()` una connessione
 - Quando accetta una connessione il processo/thread principale crea un processo/thread dedicato per il controllo e si mette in attesa di altro
- Alternative:
 - Usare opzioni per le socket per impostare un timeout
 - Usare socket non bloccante con funzione `fcntl` (funzione di gestione fd)

Socket

- **Opzioni socket**

- Opzione SO_SNDTIMEO, SO_RCVTIMEO

```
int getsockopt(int fd, level, option, void *val, socklen_t len);  
int setsockopt(int fd, level, option, void *val, socklen_t *len);
```

- opzioni di livello socket
- impostano un timeout per le operazioni di lettura/scrittura
- terminato il timeout, le operazioni di lettura/scrittura vengono interrotte con valore di ritorno negativo (errore) e `errno == EWOULDBLOCK` (o `EAGAIN`)

Socket

- **Opzioni socket**

- Opzione SO_SNDTIMEO, SO_RCVTIMEO

- **Esempio:**

```
struct timeval tv;  
tv.tv_sec = 5;    // 5 secondi  
tv.tv_usec = 0;
```

```
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));  
setsockopt(fd, SOL_SOCKET, SO_SNDTIMEO, &tv, sizeof(tv));
```

- **Effetto:**

- recv() aspetta al massimo 5 secondi
- send() aspetta al massimo 5 secondi
- Se scade → errore + errno = EWOULDBLOCK (o EAGAIN) //equivalenti

Socket

■ Opzioni socket

- Opzione SO_SNDTIMEO, SO_RCVTIMEO

```
int s = socket(AF_INET, SOCK_STREAM, 0);
if (s < 0) { perror("socket"); return 1; }

/* ---- Imposta timeout di ricezione = 5 secondi ---- */
struct timeval tv;
tv.tv_sec = 5; // secondi
tv.tv_usec = 0; // microsecondi

if (setsockopt(s, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv)) < 0) {
    perror("setsockopt RCVTIMEO");
}

/* ---- Imposta timeout di invio = 5 secondi ---- */
if (setsockopt(s, SOL_SOCKET, SO_SNDTIMEO, &tv, sizeof(tv)) < 0) {
    perror("setsockopt SNDTIMEO");
}
```

Socket

- **Opzioni socket**

- Opzione SO_SNDTIMEO, SO_RCVTIMEO

- **Esempio:**

```
#include <sys/types.h>      // tipi di base (opzionale ma tradizionale)
#include <sys/socket.h>      // socket(), bind(), connect(), send(), recv(), setsockopt()
#include <netinet/in.h>      // struct sockaddr_in, htons(), htonl()
#include <arpa/inet.h>        // inet_pton(), htons(), htonl()
#include <unistd.h>           // close()
#include <errno.h>            // errno, EAGAIN, EWOULDBLOCK
#include <string.h>           // memset(), strlen()
#include <stdio.h>            // printf(), perror()
```

Socket

- **Opzioni socket**

- Opzione SO_SNDTIMEO, SO_RCVTIMEO

- **Esempio:**

```
ssize_t n = recv(fd, buf, sizeof(buf), 0);

if (n < 0) {
    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        printf("recv() timeout: ritento...\n");
        /* decide cosa fare */
        return 0; // o ripeti
    } else {
        perror("recv");
        close(fd); // errore serio → chiude
        return -1;
    }
}
```

Socket

- **Opzioni receive_all (con possibilità di timeout)**

```
ssize_t recv_all(int fd, void *buf, size_t n) {
    size_t received = 0; char *p = (char *)buf;

    while (received < n) {
        ssize_t r = recv(fd, p + received, n - received, 0);
        if (r > 0) {
            received += (size_t)r;
            continue;
        }
        if (r == 0) {
            return (ssize_t)received; // EOF: peer ha chiuso
        }
        if (errno == EINTR) // r < 0
            continue; // interrotto da segnale: riprova
        if (errno == EAGAIN || errno == EWOULDBLOCK)
            return -2; // timeout / would-block
        return -1; // altro errore
    }
    return (ssize_t)received; // == n
}
```

Socket

- **Opzioni send_all (con possibilità di timeout)**

```
ssize_t send_all(int fd, const void *buf, size_t n) {
    size_t sent = 0; const char *p = (const char *)buf;

    while (sent < n) {
        ssize_t w = send(fd, p + sent, n - sent, 0);
        if (w > 0) {
            sent += (size_t)w;
            continue;
        }
        if (w == 0) {
            return -1; // Raro per send()
        }
        if (errno == EINTR) // w < 0
            continue; // interrotto da segnale: riprova
        if (errno == EAGAIN || errno == EWOULDBLOCK)
            return -2; // non può scrivere ora (timeout / non-blocking)
        return -1; // altro errore
    }
    return (ssize_t)sent; // == n
}
```

Buffer output/input

- Le socket TCP hanno un buffer per l'output e per l'input in cui vengono collocati temporaneamente i dati da trasmettere
- La dimensione di questo buffer può essere configurata mediante l'opzione `SO_SNDBUF`, `SO_RCVBUF`
- Se si chiama **`write()/send()`** con n byte sul socket TCP, il kernel:
 - cerca di copiare n byte sul send buffer del socket.
 - Se il buffer del socket è più piccolo di n byte o parzialmente occupato da dati non ancora trasmessi verranno copiati un numero minore di byte (short write)
 - se la socket è bloccante, può aspettare che si liberi spazio e poi copiare
 - se la socket è non-bloccante, ritorna `EAGAIN` se non c'è spazio

Buffer output/input

- Le socket TCP hanno un buffer per l'output e per l'input in cui vengono collocati temporaneamente i dati da trasmettere
- La dimensione di questo buffer può essere configurata mediante l'opzione `SO_SNDBUF`, `SO_RCVBUF`
- Chiamando **`read()/receive()`** con n bite sul socket TCP:
 - Copia n byte dal receive buffer (kernel) del socket al buffer dell'app
 - Può leggere un numero minore di byte (short read)
 - Se bloccante si blocca solo se non ci sono byte disponibili, altrimenti restituisce quelli presenti fino agli n richiesti
 - Se non bloccante restituisce subito quelli sul buffer del kernel, se nessun dato dà -1 (errore) con `errno = EAGAIN` (o `EWOULDBLOCK`)

Socket

- **Socket non bloccante**

- setsockopt() non c'è POSIX

- **Metodo comune: POSIX**

- Si modifica il file descriptor con fcntl

```
#include <fcntl.h>
```

```
int flags = fcntl(fd, F_GETFL, 0);  
fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

- **Effetto:**

- recv() non blocca → restituisce -1, errno = EAGAIN se non ci sono dati
- send() non blocca → restituisce -1, errno = EAGAIN se non c'è spazio
- funziona su TCP/UDP/pipe, ecc.

sd è il socket precedentemente aperto; F_SETFL significa che si vuole impostare il *file status flag* al valore dall'ultimo parametro; O_NONBLOCK costante corrispondente al valore che significa non bloccante.

Socket

- **Socket non bloccante**

- setsockopt() non c'è POSIX

- **Metodo comune: POSIX**

- Si modifica il file descriptor con fcntl

```
#include <fcntl.h>
```

```
int flags = fcntl(fd, F_GETFL, 0);  
fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

- **Effetto:**

- recv() non blocca → restituisce -1, errno = EAGAIN se non ci sono dati
- send() non blocca → restituisce -1, errno = EAGAIN se non c'è spazio
- funziona su TCP/UDP/pipe, ecc.

Socket

- **Socket non bloccante**

- setsockopt() non c'è POSIX

- **Esempio**

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
if (fd < 0) {
    perror("socket");
    return 1;
}

/* ---- Rendo la socket NON BLOCCANTE ---- */
int flags = fcntl(fd, F_GETFL, 0);
if (flags == -1) {
    perror("fcntl(F_GETFL)");
    close(fd);
    return 1;
}
if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0)
{
    perror("fcntl(F_SETFL)");
    close(fd);
    return 1;
}
```

Socket

■ Buffer TCP – SO_SNDBUF / SO_RCVBUF

- Ogni socket TCP ha due buffer nel kernel:
 - send buffer (uscita → rete)
 - receive buffer (rete → applicazione)
 - La loro dimensione può essere configurata tramite:
 - SO_SNDBUF → dimensione buffer di uscita
 - SO_RCVBUF → dimensione buffer di ingresso

■ Metodo:

`setsockopt(fd, SOL_SOCKET, SO_SNDBUF, &size, sizeof(size))`

`setsockopt(fd, SOL_SOCKET, SO_RCVBUF, &size, sizeof(size))`

Opzione

SO_SNDBUF

SO_RCVBUF

A cosa serve

Permette di accodare più dati al kernel → meno blocchi in `send()`

Permette di ricevere burst di dati senza perdita → riduce zero-window

Socket

- **SO_KEEPALIVE – TCP Keepalive a livello kernel**

- Permette al kernel di verificare se l'estremità remota è ancora raggiungibile quando non ci sono scambi di dati per lungo tempo
- Se la connessione rimane inattiva per un lungo periodo → il kernel invia periodicamente pacchetti “probe”
- Se non risponde → la connessione viene considerata rotta

- **Metodo:**

```
int on = 1;  
setsockopt(fd, SOL_SOCKET, SO_KEEPALIVE, &on, sizeof(on));
```

Intervalli predefiniti molto lunghi
(tipico Linux: ~2 ore di idle prima del primo keepalive)
NON garantisce un timeout rapido

Socket

- **SO_LINGER – comportamento della socket su close()**

- Controlla cosa succede quando si chiama close() mentre ci sono dati non ancora trasmessi.

- **Metodo:**

```
struct linger opt = { .l_onoff = 1, .l_linger = 5 };  
setsockopt(fd, SOL_SOCKET, SO_LINGER, &opt, sizeof(opt));
```

Dove:

`l_onoff = 1` → `SO_LINGER` attivo

`l_linger = X` → timeout X secondi

- **Comportamento:**

- Se `l_onoff = 0`, `close()` → ritorna subito (default)
→ il kernel tenta di inviare i dati rimanenti in background
- Se `l_onoff = 1`, `l_linger = X`, `close()` → BLOCCA fino a X secondi, se fallisce RST (errore al ricevente `errno = ECONNRESET`)
- Se `l_onoff = 1`, `l_linger = 0`, `close()` → immediato RST, dati scartati